

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное  
учреждение высшего образования  
Национальный исследовательский Нижегородский государственный  
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

## **Отчет по лабораторной работе**

### **«Реализация метода Гаусса для квадратной матрицы»**

**Выполнил:**

студент группы 3825Б1ПМиИ1  
Соснов С. С.

**Проверила:**

Бусько П.В.

Нижний Новгород  
2026

## Содержание

|                                      |    |
|--------------------------------------|----|
| Введение.....                        | 3  |
| Постановка задачи.....               | 4  |
| Руководство пользователя.....        | 5  |
| Описание программной реализации..... | 7  |
| Результаты экспериментов.....        | 13 |
| Заключение.....                      | 16 |
| Литература.....                      | 17 |
| Приложение.....                      | 18 |

## Введение

Система из  $n$  линейных уравнений с  $n$  неизвестными записывается в виде:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ \dots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n \end{cases}$$

В матричной форме такая система записывается как:

$$Ax = b$$

Где  $A$  – матрица коэффициентов этой системы,  $b$  – вектор правой части системы, а  $x$  – вектор, который нужно найти. Такие системы возникают довольно часто, поэтому нужен метод решения этих систем.

Метод решения систем линейных уравнений называется метод Гаусса или метод исключения неизвестных. Метод Гаусса строит последовательность матриц эквивалентных линейных систем уравнений. В комментариях к вычислениям используется термин *ведущий элемент*, строка и столбец, содержащие ведущий элемент, называются *ведущей строкой* и *ведущим столбцом*. Каждый этап вычислений состоит из трёх шагов:

- В качестве ведущей строки выбираем первую ненулевую строку.
- В качестве ведущего элемента выбираем первый слева ненулевой элемент ведущей строки.
- Элементарными преобразованиями строк зануляем все элементы ведущего столбца, кроме ведущего элемента.

Элементы матрицы коэффициентов могут быть вещественными, но при реализации этого метода на компьютере, элементы становятся числами с плавающей точкой, что вносит погрешность округления, при большом количестве операций ошибки накапливаются, что может полностью исказить результат. Следовательно, реализовывать алгоритм нужно по-другому, чтобы минимизировать ошибки округления.

## Постановка задачи

Цель:

Реализовать метод Гаусса для квадратной матрицы с выбором ведущего элемента на языке Си++.

Необходимо:

- Реализовать собственный шаблонный класс вектор.
- Реализовать шаблонный класс квадратной матрицы, которая является потомком класса вектора от вектора.
- Реализовать класс СЛУ, который является потомком квадратной матрицы и имеет метод Гаусса.
- Реализовать метод Гаусса так, чтобы он верно обрабатывал случаи, если матрица коэффициентов вырождена, или числа в СЛУ подобраны так, что дальнейшая работа алгоритма даст неточный ответ.
- Реализовать возможность работы с одной и той же матрицей коэффициентов, меняя только правый вектор.

## Руководство пользователя

Данная программа позволяет пользователю построить систему уравнений, дав сначала ввести квадратную матрицу коэффициентов для этой системы, а потом вектор правой части. После вывода ответа, пользователь сможет продолжить работу с этой же матрицей, нужно будет только заново ввести правую часть.

1. При запуске программы пользователя попросят ввести размер квадратной матрицы коэффициентов (Рис.1.). Необходимо *ввести желаемый размер матрицы*.

```
Enter size of square matrix (positive integer):  
3
```

Рис.1. Пример ввода желаемой размерности.

2. После пользователя попросят заполнить эту матрицу числами (Рис.2.). Необходимо ввести все элементы через пробел. (Если количество чисел будет избыточно, программа примет только первые значения, остальные отбросит).

```
Fill this matrix with numbers:  
1 -8 04 3 87 491 -19 9 87 41 1 4 321 1
```

Рис.2. Пример ввода чисел для заполнения матрицы (с избытком).

3. Затем пользователь должен будет ввести координаты вектора правой части системы (Рис.3.). Необходимо ввести все числа через пробел. (Если количество чисел будет избыточно, программа примет только первые значения, остальные отбросит).

```
Enter the coordinates of the vector of the right part of the system:  
-98 8 92 1 84 0 1 3
```

Рис.3. Пример ввода чисел для заполнения вектора (с избытком).

4. Далее программа выведет в консоли систему, которую ввёл пользователь и ответ (Рис.4.).

```
The system introduced:  
  
1 -8 4 = -98  
3 87 491 = 8  
-19 9 87 = 92  
  
Answer:  
  
x1 = -8.04061  
x2 = 10.3598  
x3 = -1.77023
```

Рис.4. Пример ввода системы и ответа для ранее вводимой системы.

5. Наконец, пользователю предложат продолжить работу с текущей матрицей коэффициентов, если согласиться, то нужно будет заново ввести вектор правой части, если отказаться – программа закончит работу (Рис.5.). Необходимо *ввести Y или y* для продолжения работы, *N или n* – для завершения работы.

```
Do you want to continue working with this coefficient matrix? (Y/n)  
Y  
Enter the coordinates of the vector of the right part of the system:  
|
```

Рис.5. Пример ввода для продолжения работы .

## Описание программной реализации

Проект состоит из 6 файлов, 4 заголовочных файла, 1 главный файл программы и 1 .exe файл.

В данной работе реализован метод Гаусса – Жордана, то есть матрица коэффициентов приводится не треугольному виду, а к единичному. Программа может обрабатывать случаи, когда матрица коэффициентов вырождена, или числа системы такие, что при решении, алгоритм не может гарантировать заданную точность ответов, выдавая об этом сообщение пользователю. Выбор ведущего элемента выполнен таким образом, чтобы попытаться минимизировать ошибки округления при вычислении.

### Vector.h

Файл `Vector.h` содержит реализацию шаблонного класса «вектор». Использование шаблонов позволяет создавать векторы, координаты которых имеют разные типы данных, без написания доп. кода. Во всех экспериментах использовалась программа, где координаты имеют тип `double`, изменить это можно, поменяв одно слово в главном файле и перекомпилировав программу заново. Класс имеет два `protected` поля:

- ~ `size` – количество координат вектора (размерность вектора).
- ~ `coordinates` – указатель на массив координат вектора.

Класс имеет 16 методов: 3 конструктора, 8 перегруженных операций, 4 обычных метода и один деструктор. Такое количество методов позволяет удобно работать с векторами в алгоритме. Все методы `public`:

- ~ `Vector()` – конструктор по умолчанию, присваивает нулевой указатель массиву координат, а размеру 0.
- ~ `Vector(unsigned int size)` – конструктор инициализатор, присваивает размеру переданное число и выделяет память под `size` координат в куче.
- ~ `Vector(const Vector& v)` – конструктор копирования, копирует данные переданного вектора в новый вектор.
- ~ `InputVector()` – метод, который позволяет ввести координаты вектора. Ничего не возвращает.
- ~ `PrintVector() const` – выводит циклом все координаты вектора. Ничего не возвращает.
- ~ `GetSize() const` – метод, который возвращает значение поля `size` вектора.

~ `GetCoor() const` – метод, который возвращает указатель на массив координат вектора.

~ `operator*= (const Vector& rhs)` – поэлементно выполняет операцию `*=` для координат исходного вектора, и переданного. Ничего не возвращает.

~ `operator-= (const Vector& rhs)` – поэлементно выполняет операцию `-=` для координат исходного вектора, и переданного. Ничего не возвращает.

~ `operator== (const Vector& rhs) const` – сравнивает размеры и координаты векторов, возвращает `true`, если все величины равны, `false` – если нет.

~ `T& operator[] (unsigned int i)` – возвращает `i`-ую координату вектора.

~ `const T& operator[] (unsigned int i) const` – `operator[]` для констант.

~ `operator* (const T& multiplier) const` – возвращает новый вектор, где все координаты умножены на `multiplier`.

~ `operator/ (const T& divider) const` – возвращает новый вектор, где все координаты поделены на `divider`.

~ `operator= (const Vector& rhs)` – присваивает вектору, значения переданного вектора, правильно распределяя память.

~ `~Vector()` – деструктор, удаляет массив координат в памяти.

Так как данный класс управляет памятью, то код написан по правилу трёх, то есть реализованы: деструктор, конструктор копирования, оператор присваивания. Правильная реализация этих методов позволяет не испытывать проблем с памятью.

## Matrix.h

Файл `Matrix.h` содержит реализацию шаблонного класса квадратичной матрицы, который наследуется от «вектора векторов» (`Vector<Vector<T>>`). Этот тип позволяет работать с матрицей коэффициентов, что нужно в реализации алгоритма. Класс имеет только четыре `public` метода: 1 конструктор и 3 метода:

~ `Matrix(unsigned int n)` – конструктор инициализатор, который присваивает полю `size n` и выделяет память в куче для всех элементов матрицы.

~ `PrintMatrix() const` – метод, который циклом выводит все элементы матрицы в правильном порядке. Ничего не возвращает.

~ `InputMatrix()` – метод, который позволяет переопределить элементы матрицы (ввод идёт построчно). Ничего не возвращает.



~ SwapLine(int i, int j) – метод, который принимает номера строки матрицы и, проверив отличие строк, меняет их местами. Ничего не возвращает.

## Exceptions.h

Файл Exception.h содержит только одну структуру: struct Exception, которая имеет два поля:

~ kod – код исключения. Беззнаковое целое.  
~ message – массив символов, сообщение, которое описывает суть исключения.

Данная структура была сделана для удобной работы с исключениями, а точнее для обработки.

## SLU.h

Файл SLU.h представляет собой реализацию шаблонного класса системы линейных уравнений (СЛУ), данный класс наследуется от класса квадратичной матрицы, т.к. СЛУ имеет матрицу коэффициентов, а при решении этой системы активно используются элементарные преобразования над строками и столбцами матрицы. Класс имеет только четыре метода: 1 конструктор и три обычных метода, все методы public:

~ SLU(unsigned int n) – вызывает конструктор матрицы, чтобы выделить память на матрицу коэффициентов.  
~ PrintSLU(const Vector<T>&b) const – выводит СЛУ, принимая правый вектор. Ничего не возвращает.  
~ PrintAnswer(const Matrix<T>& E, const Vector<T>& ans) const – в программе используется только при выводе ответа, принимает единичную матрицу и вектор с ответами, выводя значение каждого корня. Ничего не возвращает.  
~ GaussMethod(Vector<T>& right, const T& epsilon) – принимает вектор правой части и epsilon для регулирования точности ответа (подробнее на стр. 11 и 15), решает систему, вызывая в конце функцию PrintAnswer, чтобы вывести ответ. Ничего не возвращает.

В главном файле GaussianMethod.cpp написана реализация самой программы, возможность задать желаемую СЛУ и продолжить работу с той же матрицей коэффициентов, там же реализована функция ClearBuffer(), которая очищает буфер, чтобы правильно обрабатывать случаи, когда пользователь вводит данных больше чем нужно.

Также в проекте есть файл `Solution_SLU.exe`, сделанный для более удобного проведения экспериментов. В этой скомпилированной программе тип элементов матрицы является `double`, а точность вычислений `epsilon=1e-13`, для изменения этих параметров нужно менять главный файл и заново компилировать.

## Разбор алгоритма

Т.к. метод Гаусса является ключевым в данной программе, то нужно разобрать его работу поподробней. Для объяснения работы алгоритма приведён псевдокод:

Вход: вектор `right` правой части системы размерности `n` (система уже задана матрицей коэффициентов `A`), порог точности `e`.

Выход: решение системы (вывод координат вектора `ans`) или сообщение о вырожденности или о недостижимости точного ответа.

1. Создание временной матрицы `tmp`

2. **Для** столбцов от `j = 0` до `n - 1`

3. **Для** строк от `i = j` до `n - 1`

Найти номер с макс. элементов в столбце `j` (исключая ведущую строку).

`maxrow`  $\leftarrow$  номер строки с максимальным элементом в `j`-ом столбце

а) Проверить на случаи вырожденности матрицы или недостижимости точного ответа.

**Если** `tmp[maxrow][j] = 0`

**то** вернуть пользователю сообщение о вырожденности матрицы коэффициентов

**Если** `|tmp[maxrow][j]| < e`

**то** вернуть пользователю сообщение о том, что решение недостижимо для текущего порога точности `e`

б) Нормировать всю ведущую строку включая правый вектор.

```
right[maxrow] <- right[maxrow] / tmp[maxrow][j]
```

```
tmp[maxrow] <- tmp[maxrow] / tmp[maxrow][j]
```

с) Поставить ведущий элемент на диагональ.

```
right[j] <-> right[maxrow]
```

```
tmp[j] <-> tmp[maxrow]
```

д) Сделать ведущий элемент положительным.

**Если** tmp[j][j] < 0

```
    то right[j] <- right[j] * (-1)
```

```
    tmp[j] <- tmp[j] * (-1)
```

е) Занулить все элементы в столбце j, кроме ведущего.

**Для** всех i от 0 до n - 1

```
    right[i] <- right[i] - right[j] * tmp[i][j]
```

```
    tmp[i] <- tmp[i] - tmp[j] * tmp[i][j]
```

4. **Вызвать функцию** PrintAnswer(tmp, right), которая напечатает координаты вектора ans.

При работе алгоритма, ведущей строкой выбирается именно та, у которой максимальный элемент в j-ом столбце, а не первая ненулевая, как в оригинальном методе, также выбор ведущего элемента идёт строго по столбцам, так как программа не может правильно вычислить ответ для системы, где матрица коэффициентов вырождена. Выбор максимального элемента нужен для того, чтобы программа вычисляла ответ с минимальной погрешностью, так как если окажется, что ведущий элемент будет очень мал, тогда все остальные числа в строке, при нормировании вектора строки, станут огромными, после зануления всего ведущего столбца, кроме самого ведущего элемента, в остальных строках также появятся огромные числа; при дальнейшей работе программы, эти числа также будут складываться или вычитаться, но настолько большие числа не смогут точно храниться, из-за чего возникнет ошибка округления, далее ошибка будет копиться до конца работы программы. Именно поэтому ведущий элемент сравнивается с  $\epsilon$ , так как если максимальный элемент настолько мал, то дальше при работе программы накопятся ошибки, а ответ станет неточным; регулируя значение  $\epsilon$ , можно регулировать поведение программы: насколько точный ответ она сможет дать. Именно такой алгоритм минимизирует влияние ошибок округлений и позволяет

обрабатывать случаи, когда ответ может быть неточным или при работе программы появляется вырожденная матрица коэффициентов.

## Результаты экспериментов

В ходе работы были проведены эксперименты для проверки корректности работы программы, анализировались решения для заданных СЛУ, уделялось внимание точности решения. Эксперименты были проведены для совместной СЛУ с единственным решением, для СЛУ, у которой матрица коэффициентов вырождена, а также рассматривались случаи, когда числа подобраны так, что точный ответ не гарантирован.

### Совместная СЛУ

Для проверки корректности алгоритма, были рассмотрены две совместные СЛУ с единственным решением.

$$\begin{cases} x_1 + x_2 + x_3 = 3 \\ x_1 - x_2 + x_3 = 1 \\ 2x_1 + x_2 - x_3 = 2 \end{cases}$$

И

$$\begin{cases} 2x_1 + x_2 + x_3 + x_4 = 11 \\ x_1 + 3x_2 + 2x_3 + x_4 = 17 \\ x_1 + x_2 + 4x_3 + 3x_4 = 27 \\ 3x_1 + 2x_2 + x_3 + 2x_4 = 18 \end{cases}$$

В результате работы программы были получены ответы:

$$x_1 = 1 \quad x_2 = 1 \quad x_3 = 1$$

И

$$x_1 = 1 \quad x_2 = 2 \quad x_3 = 3 \quad x_4 = 4$$

Данные ответы являются верными, следовательно, алгоритм для совместных систем с единственным решением работает верно и базовая логика программы не нарушена.

### СЛУ с вырожденной матрицей коэффициентов

Для проверки корректности обработки СЛУ с вырожденной матрицей коэффициентов были рассмотрены те же системы, что и в прошлом эксперименте, но изменённой матрицей коэффициентов так, чтобы строки были линейно зависимы:

$$\begin{cases} 8x_1 + 4x_2 - 4x_3 = 3 \\ x_1 - x_2 + x_3 = 1 \\ 2x_1 + x_2 - x_3 = 2 \end{cases}$$

Здесь видно, что первая строка получена из второй умножением её на два. Данная система несовместна, так как  $\text{rank}(A) = 2$ , а  $\text{rank}(A') = 3$ , где  $A$  – матрица коэффициентов, а  $A'$  – расширенная матрица коэффициентов, следовательно и решений у неё нет. Для данной системы программа выведет сообщение *“Zero column, matrix is singular => either there are no solutions, or there are infinitely many of them”*. То есть после нескольких элементарных преобразований получился нулевой столбец, следовательно, система либо не имеет решений, либо их бесконечно много. В данном случае решений нет. Для системы:

$$\begin{cases} 2x_1 + 6x_2 + 4x_3 + 2x_4 = 34 \\ x_1 + 3x_2 + 2x_3 + x_4 = 17 \\ 6x_1 + 4x_2 + 2x_3 + 4x_4 = 36 \\ 3x_1 + 2x_2 + x_3 + 2x_4 = 18 \end{cases}$$

Видно, что первая и третья строки получены путём умножения второй и четвёртой на 2, но в отличие от первой системы, данная – совместна, но имеет бесконечно много решений. При вводе этой системы, получим то же сообщение *“Zero column, matrix is singular => either there are no solutions, or there are infinitely many of them”*. Здесь также в ходе работы программы получился нулевой столбец, т.к.  $|A| = 0$ , но программа не различает совместные и несовместные системы, она смотрит только на матрицу коэффициентов, от чего может давать ответ в виде вектора только для систем с единственным решением.

### **Случай, когда точного решения нет.**

Теперь рассмотрим систему, при вычислении которой алгоритм не сможет дать точный ответ.

$$\begin{cases} 2x_1 + 2x_2 = 4 \\ 2x_1 + (2 + 10^{-14})x_2 = 4 + 10^{-14} \end{cases}$$

Очевидно, что ответ здесь:

$$x_1 = 1 \quad x_2 = 1$$

Но программа выдаёт сообщение *“For a given system, with a given error bound epsilon, the algorithm cannot find a sufficiently accurate answer”*. То есть программа не может дать точный ответ при заданном  $\epsilon$ , который в скомпилированной программе равен  $1e-13$ . Но если понизить  $\epsilon$  до  $1e-15$ , то программа уже сможет выдать результат:

$$x_1 = 1.91304 \quad x_2 = 0.0869565$$

Он неверный, но при подстановке координаты правого вектора будут 3.999993 и 3.999993, что почти верно. То есть  $\epsilon$  выступает регулятором точности, при понижении степени у 10, программа начинает сравнивать ведущий элемент с числом, которой всё ближе к нулю, из-за чего точность полученного ответа снижается, данный пример это отлично демонстрирует, если же нужен более точный ответ, то нужно увеличивать значение  $\epsilon$ .

## Итоги экспериментов

Проведённые эксперименты показали, что алгоритм работает верно для любых систем, он верно обрабатывает случаи, когда матрица коэффициентов вырождена, или точный ответ получить нельзя. Но результаты решения зависят не только от алгоритма, но и от самой системы, иногда получить точный ответ при текущем алгоритме невозможно.

## **Заключение**

В ходе выполнения работы был реализован метод Гаусса-Жордана, алгоритм был реализован так, чтобы минимизировать ошибки округления при вычислении. Также были разработаны собственные шаблонные классы. Было исследовано влияние порога  $\epsilon$  на конечный результат работы программы, а также было объяснено, почему выбор ведущего элемента критически важен для устойчивости.



## **Литература**

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.

## Приложение

```
#define EPS 1e-13
#include "Vector.h"
#include "SLU.h"
#include <stdio.h>
#include "Exceptions.h"
#include <iostream>

void ClearBuffer()
{
    int c;
    while ((c = getchar()) != '\n');
}

typedef double type;
const type epsilon = EPS;

int main()
{
    unsigned int n;

    do
    {
        printf("Enter size of square matrix (positive
integer):\n");
        scanf("%u", &n);
        ClearBuffer();
    } while (n <= 0);

    SLU<type> Sys(n);

    printf("Fill this matrix with numbers:\n");
    Sys.InputMatrix();
    ClearBuffer();

    char choice;
    do
    {
        Vector<type> right(n);

        printf("Enter the coordinates of the vector of the
right part of the system:\n");
        right.InputVector();
        ClearBuffer();

        printf("The system introduced:\n\n");
        Sys.PrintSLU(right);

        printf("Answer:\n\n");

        try
        {
```

```

        Sys.GaussMethod(right, epsilon);
    }
    catch (const Exception& e)
    {
        switch (e.kod)
        {
            case 1: std::cout << e.message << '\n'; break;
            case 2: std::cout << e.message << '\n'; break;
            case 3: std::cout << e.message << '\n'; break;
            case 4: std::cout << e.message << '\n'; break;
        }
    }
    do
    {
        printf("Do you want to continue working with
this coefficient matrix? (Y/n) \n");
        scanf("%c", &choice, sizeof(choice));
        ClearBuffer();
    } while (choice != 'Y' && choice != 'y' && choice !=
'N' && choice != 'n');
    } while (choice == 'Y' || choice == 'y');
    return 0;
}

```